CSE-1st Semester

Programing with C language.

→ C, computer programming language developed in the early 1970s by American computer scientist Dennis M. Ritchie.

- **Source code:** Source code is a collection of instructions written in a programming language that a computer can understand and execute. It serves as the human-readable version of a program, allowing developers to write, modify, and maintain software.

- **Object code:** Object code is the machine-readable output generated from source code after it has been compiled. It consists of binary instructions that a computer's processor can execute directly. Object code is typically not human-readable, as it is in a format specific to the target machine's architecture.

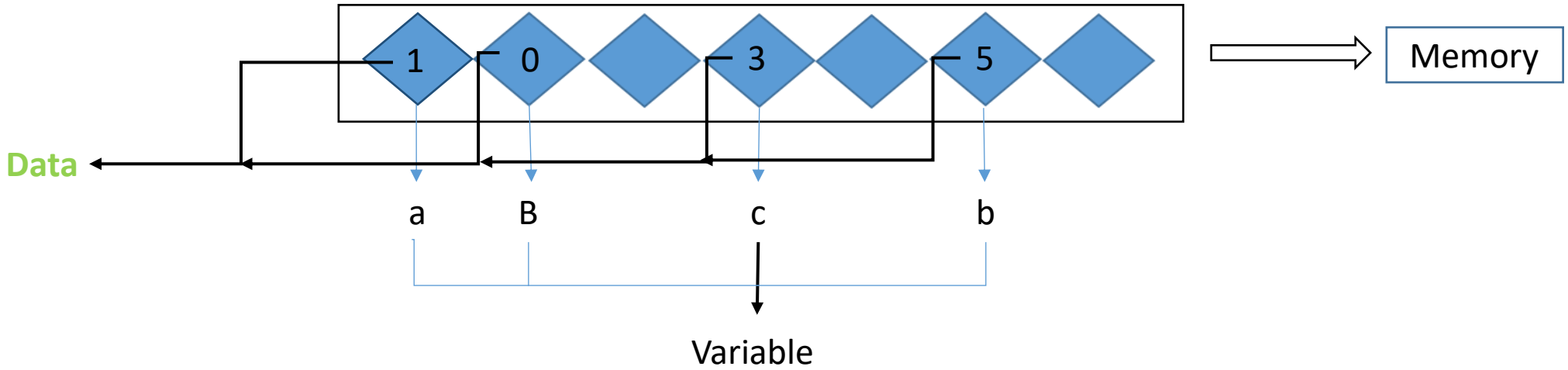
- **Translator program:** A translator program is software that converts code written in one programming language into another. There are several types of translator programs: Compiler, Interpreter, Assembler etc.

  - **Compiler:** Translates source code (high-level language) into machine code (object code) in one go, producing an executable file. Examples include GCC for C/C++ and javac for Java.
  - **Interpreter:** Translates and executes source code line-by-line or statement-by-statement at runtime, without producing a separate executable file. Examples include Python's CPython.
  - **Assembler:** Converts assembly language code into machine code. Assemblers translate low-level code that is closely related to the architecture of the computer.

- **Programing symbols:**

Normal symbol	C operators
+	+
-	-
x	*
÷	/

❖ **Variables:** It is a name of a memory location which stores some data.



- Variables are case sensitive.(b and B are not same)
- 1st character should be alphabet or '_' (Underscore).
- No comma , blanks, dot or any other special character can not be used (* '_').
- Variables can change by statements.
- Keyword(int, double, else, long, short, continue, default, const, do, sizeof, goto, switch, for etc.) can't be used as a variable.

- Variable type:

Integer	Fraction		Character	
int	float	double	char	
%d	%f	%lf	%c	%s

Format specifier

Single cote(' ') — For 1 letter

For multiple letter — Double cote(" ")

Data type	Byte
char	1
int	2
long int	4
float	4
double	8
long double	10

Program structure:

```
#include<stdio.h>
int main(){

    return 0;

}
```

Header file / Preprocessor directive.

Main function; all process starts after this.

- **Line break:** ‘ \n ’.
- **Input:** ‘ scanf(“% d”,&s); ’
- **Output:** ‘ printf(“__ %d”, s); ’

❖ **Comments:** Lines that are not part of program.

- Single line comments.(//comment)
- Multiple line comments.(/*comments*/)

❖ **Operators:**

- Arithmetic operator: +, -, *, /, %
- Relational operator: ==, >, >= , <, <= , !=
- Logical operator: &&, ||, !
- Assignment operator: =, +=, -=, *=, /=, %= (a =a%b ⟹ a%=b)
- Bitwise operator: Operator that is used for bit number(0,1).
- Ternary operator: Also called as conditional operator, is used instead of “if….else”.

Doing short. (Assign. ope. Is also known as Short hand ope.)

❖ **Operators Precedence**:(Priority wise)

Decreases

!
*,/,%
+,-
<,<=,>,>=
==,!=
&&
||
=

Associativity:(For same line)

→ Left to Right.

- 4*3/6*3 = 12/6*2 = 2*2 = 4 .
- (4*3)/(6*3) = 12/18 = 0 .(0.67)
- 5+2/2*3 = 5+1*3 = 5+3 = 8 .
- 5*2-3*2 = 10-6 = 4 .

❖ **Instructions:** Statements in a program.

- Type declaration instructions.
- Arithmetic instructions.
- Control instructions.

❖ **Type declaration:** (Always declare variables before using it)

(=) it is an assignment operator. It assign a value to variables.

Right example:

1.
```
int a = 25;
int b = 2* a;
```

2.
```
int a,b,c;
a=b=c=1
```

Wrong example:

1.
```
int a = 25;
int c = a*b;
int b = 22;
```
A variable should first declare than use.

2.
```
int a=b=c=1;
```
For multi-assign ,first declare than multi-assign. Declaration and multi-assign can't be used parallelly.

❖ Arithmetic instructions:

x = a + b;

Result (Single variable)

Operand/ Variables(As much as u like)

Valid :

a = b + c;
a = b * c;
a = b + c + d;

Invalid :

a = b c;
b + c = a;
a = b (c)^-1;

❖ Type conversion:

int	OP	int	= int
int	OP	double	= double
float	OP	float	= float

Type Cast : (A part of type conversion)
(int) 1.25562 = 1
(double) 1= 1.00000000
->(type) variable or number = result will be converted in the type.

❖ Control instructions: ->Determine flow of program.

- Sequence control : Sequence wise write.
- Decision control : if..else.
- Loop control : for, while, do…while.
- Case control : Case dec. wise (what to do and not to do).

- Heder file
- Type dec.
- Variable dec.
- Process

Practice some examples.. Using operators..

❖ Conditional statements:

If….else :

```
If(condition){
        process(if the condition is true)
        }
else{
        process(if the condition is false)
        }
```

```
int a=10;
int b=7;
if(a>b){
        printf("A is big");
        }
else{
        printf("B is big");
        }
```

If…else if :

In case 1st if is true than else if and else won't be check.

```
if(condition1){
        process(if condition1 is true)
        }
else if(condition2){
        process(if condition1 is false and condition2 is true)
        }
else{
        process(if condition2 is false)
        }
```

```
int a=10;
int b=70;
if(a>b){
   printf("A is big");
   }
else if(b>a){
   printf("B is big");
   }
else {
   printf("A and B is equal");
   }
```

Multiple if :

This time all if will be check.

```
if(condition1){
        process(if condition1 is true)
        }
if(condition2){
        process(if condition2 is true)
        }
if(condition3){
        process(if condition3 is true)
        }
```

```
int a=10;
int b=20;
if(a>0){
   printf("A is positive\n");
   }
if(b>0){
   printf("B is positive\n");
   }
if((b%2==0)&&(a%2==0)){
   printf("A and B is even\n");
   }
```

Nested if…else : (if…else inside if)

In case 1st if is false than if..else inside 1st if won't be check but else and if..else inside else will be check. On the other hand in case the 1st if is true than ,if..else inside 1st if will be check and else and it's part won't be check.

```
if(condition1){
        (if condition1 is true)
        process;
        if(condition2){
                process(if condition2 is true)
                }
        else{
                process(if condition2 is false)
                }
```

```
else{
        (if condition1 is false)
        process;
        if(condition3){
                process(if condition3 is true)
                }
        else{
                process(if condition3 is false)
                }
```

❑ Practice nested if..else and other conditional statement..

ASCAII value : (Decimal value of character and symbols.) ('A'=65 to 'Z'=90), ('a'=97 to 'z'=122) others are for symbols.

➢ Write a program to find a character entered by user , is upper case or lower case..

Answer:

```
#include<stdio.h>

int main(){
  char character;

  printf("Enter character: ");
  scanf("%c", &character);

    if (character >= 'A'&& character <= 'Z'){
                    printf("Upper case\n");
    }

    else if (character >= 'a' && character <= 'z'){
                    printf("Lower case\n");
    }

    else printf("Not a character\n");

  return 0;
}
```

❖ **Ternary operator** : (A short form of if..else; used instead of if..else.)

Syntax :

(condition)? process(if true) : process(if false) ;

Called 'if not'

Ex:

```
#include<stdio.h>
int main(){
int a=7;
(a%2==0)? printf("Even") : printf("Odd");
 return 0;
}
```

Answer: Odd

❖ **Switch** : (Case in order)

Syntax:

Input Variable

```
switch(day){
        case 1: Process(if input is 1);
                break;
        case 2: Process(if input is 2);
                break;
        case 3: Process(if input is 3);
                break;

        default : Process(if input not available );
}
```

Breaks the switch if current case is true.

Works if all case is false.

Ex:

```
#include<stdio.h>
int main(){
 int day;
 printf("Enter day(1-7): ");
 scanf("%d", &day);

 switch(day){
     case 1:printf("monday");
            break;
     case 2:printf("tuesday");
            break;
     case 3:printf("wednesday");
            break;
     case 4:printf("thursday");
            break;
     case 5:printf("friday");
            break;
     case 6:printf("saturday");
            break;
     case 7:printf("sunday");
            break;

     default :printf("not valid");
 }
 return 0;
}
```

❖ Nested Switch : (Switch in switch)

Syntax:

```
switch (x) {
        case 1: Process(if x input is 1);
                switch (y) {
                    case 1: Process(if x input is 1 & y input is 1);
                            break;
                    case 2: Process(if x input is 1 & y input is 2);
                            break;
                }
        break;

        case 2: Process(if x input is 2);
                switch (y) {
                    case 1: Process(if x input is 2 & y input is 1);
                            break;
                    case 2: Process(if x input is 2 & y input is 2);
                            break;
                }
        break;

        default: printf( "Choice is not available");
```

Breaks the switch if the current case is true, or continues until the default or another case is true.

Ex:

```
#include <stdio.h>

int main()
{
   int x = 1, y = 2;

   switch (x) {

      case 1:  printf( "Choice x is 1");

               switch (y) {

               case 1:
                  printf( "Choice y is 1");
                  break;

               case 2:
                  printf( "Choice y is 2");
                  break;
               }
      break;

      case 2:  printf( "Choice x is 2");

               switch (y) {

               case 1:
                  printf( "Choice y is 1");
                  break;

               case 2:
                  printf( "Choice y is 2");
                  break;
               }
      break;

   default: printf( "Choice is not available");

   }
   return 0;
}
```

❖ **Loop** : 3 type. (for, while, do….while)

For loop:

Syntax:

```
for(initial statement; condition; upgrading statement){
        Process;
}
```

Infinite loop: (a program that repeats indefinitely)

Causes:

1. Incorrect Conditions.(Condition always true, No termination logic)
2. Missing Updates.(Forgetting to update variables)

Ex : (Printing number 1 to 10)

```
#include <stdio.h>
int main() {
        int i;
        for (i = 1; i < 11; ++i) {
                printf("%d ", i);
                }
        return 0;
        }
```

Output: 1 2 3 4 5 6 7 8 9 10

- **Increment operators** : 2 type.(Pre-increment , post increment)

(++i) (i++)

➢ (i++): In **post-increment**, the current value of the variable is **used first**, and then the variable is incremented.

code: printf("%d\n", i++);
printf("%d\n", i);

Output: (if i=1) ➔ 1 (i=1,i+1=2 in next line)
2

➢ (++i): In **pre-increment**, the value of the variable is **incremented first**, and then the incremented value is used in the expression.

code: printf("%d\n", i++);
printf("%d\n", i);

Output: (if i=1) ➔ 2 (i=1,i+1=2 in current line)
2

- **Decrement operators** : 2 type.(Pre-decrement, post decrement)

(--i) (i--)

Works same as Increment operators.

While loop:

Syntax:

```
initial statement or declaration
while(condition){
        Process.
        Upgrading statement.
    }
```

Ex : (Printing number 1 to 10)

```c
#include <stdio.h>
 int main() {
        int i = 1;
        while ( i < 11) {
            printf("%d ", i);
            i++;
            }
        return 0;
        }
```

Output: 1 2 3 4 5 6 7 8 9 10

Do…..while loop :

Syntax:

```
initial statement or declaration
do{
    Process.
    Upgrading statement.
  } while(condition);
```

Ex : (Printing number 1 to 10)

```c
#include <stdio.h>
 int main() {
        int i = 1;
        do {
            printf("%d ", i);
            i++;
            } while ( i < 11);
        return 0;
        }
```

Output: 1 2 3 4 5 6 7 8 9 10

- **Break statement**: The break statement is used to exit the loop immediately when a certain condition is met. It can be use in loop without conditional statement but it has no usefulness. So generally it is used in a conditional statement.

Ex:

```
for(int i=1;i<10;i++)
{ if (i== 5){
            break;
          }
printf("%d\n",i);
}
```

Output :

```
1
2
3
4
```

Stops after the condition (i=5) is true.

- **Continue statement** : The continue statement is used to skip the current iteration (when the condition is true) and proceed to the next iteration. The loop itself continues running.

Ex:

```
for(int i=1;i<10;i++)
{ if (i== 5){
            break;
          }
printf("%d\n",i);
}
```

Output :

```
1
2
3
4
6
7
8
9
```

Skips only single loop of (i=5) when condition (i=5) is true. But continue other loops.

- **Pattern printing** : It is a section that uses loop to print different types of pattern.

1. Given an integer **N**. Print the following pattern.

Sample Input	Sample Output
3	*** *** ***
4	**** **** **** ****

Answer:

```
#include<stdio.h>
int main(){
        int n,i,j;
        scanf("%d", &n);

        for(i=1 ; i<=n ; i++){
            for(j=1 ; j<=n ; j++){

                printf("* ");
            }
            printf("\n");
        }
            return 0;
    }
```

Output: 5

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

2. Given two integers **N** and **M**. Print an **N x M** square.

Sample Input	Sample Output
3 2	** ** **
3 4	**** **** ****

Answer:

```
#include<stdio.h>

int main(){
  int n,m,i,j;
  scanf("%d%d", &n,&m);

   for(i=1 ; i<=n ; i++){
      for(j=1 ; j<=m ; j++){

          printf("* ");
        }
      printf("\n");
   }
return 0;
}
```

Output: 4 3

```
* * *
* * *
* * *
* * *
```

3. Given an integer **N**. Print the following pattern.

Sample Input	Sample Output
3	* ** ***
4	* ** *** ****

4. Given an integer **N**. Print the following pattern.

Sample Input	Sample Output
3	* ** ***
4	* ** *** ****

5. Given an integer **N**. Print the following pattern.

Sample Input	Sample Output
3	* *** *****
4	* *** ***** *******

- In 3. no. ➔ for N=4 ➔

i \ j	1	2	3	4
1	*			
2	*	*		
3	*	*	*	
4	*	*	*	*

```
for(i=1 ; i<=n ; i++){
    for(j=1 ; j<=i ; j++){

        printf("* ");
    }
    printf("\n");
}
```

- In 4. no. ➔ for N=4

i \ j	1	2	3	4
1				*
2			*	*
3		*	*	*
4	*	*	*	*

```
for(i=1 ; i<=n ; i++){
    for(j=n ; j>=1 ; j--){

        if(i>=j)
            printf("* ");//*and a space

        else  printf("  ");//two space
        }

    printf("\n");
}
```

- In 3. no. ➔ for N=4 ➔

```
for(i=1 ; i<=n ; i++){
    for(j=n ; j>=1 ; j--){

        if(i>=j)
            printf("* ");//*and a space

        else  printf("  ");//two space
        }

    printf("\n");
}
```

➢ Practice this patterns.

6. Given an integer **N**. Print the following pattern.

Sample Input	Sample Output

3	*****
	  ***
	  *

4	*******
	  *****
	  ***
	  *

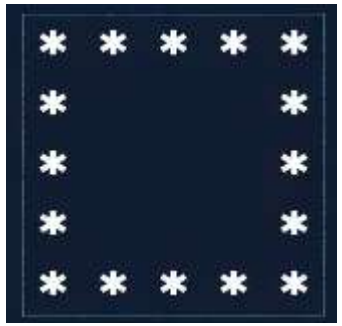
7. Given an integer **N**. Print the following pattern.

Sample Input	Sample Output

3	*
	  ***
	*****
	  ***
	  *

4	*
	  ***
	  *****
	*******
	  *****
	  ***
	  *

8. Given an integer N. Print following pattern.



9. Given an integer N. Print following pattern.

	C1	C2	C3	C4	C5
R1	*				*
R2		*		*	
R3			*		
R4		*		*	
R5	*				*

❖ **Functions** : 2 type. Library function(In built like scanf()),User defined(Declared and defined by programmer).

- Blocks of code that performs particular task.
- It can be used multiple times.
- Increase code reusability.

- Function can only return one value at a time.
- Changes to parameter in function don't change the values in calling function. (because a copy of argument is passed to the function the real value remain intact)

Syntax:

```
#include<stdio.h>
void Name(){

    printf("Samia\n");

}
int main()
{
        Name();
        Name();

        return 0;
}
```

Function declaration.

Function definition.

Function call.

Return type.. Can be void, int, float, double.This indicates the data type return values.
***Void**: Returns 0 or nothing to main function but in can work and print in it's own.(if called)

Output: Samia
Samia

Another syntax:

```
#include<stdio.h>

void Name ();

int main()
{
        Name();
        Name();

        return 0;
}
void Name(){

        printf("Samia\n");

        }
```

Function name.. Can be anything.(can't be the existing ones) Like variable.

Output: Samia
Samia

❑ Passing arguments :

➔ A function ➔ Take value (parameter)
➔ Gives value (return value)

➔ void print_Name( )
➔ void prinTable(int n)
➔ int sum(int a , int b)

parameter

return value

→ Actual parameter/Argument(In function call)

→ Formal parameter/Parameter(In function declaration)

Argument	Parameter
➔ Values that are passed in function call. ➔ Actual parameter. ➔ Used to send value.	➔ Values in function declaration and definition. ➔ Formal parameter. ➔ Used to receive value.

Ex : Print times table.

```
#include<stdio.h>

void tables(int n){
   for(int i=1;i<=10;i++){
      printf("%d X %d = %d\n", n, i, i*n);
   }
}
int main()
{  int n;
   printf("Enter number: ");
   scanf("%d", &n);

   tables(n);

   return 0;
}
```

Ex : Calculate area of square, circle and rectangle.

```
#include<stdio.h>
#include<math.h>

double square_area(double side){
   return (side*side);
   }
double circle_area(double radius){
   return (3.1416*pow(radius,2));
   }
double rectangle_area(double length,double width){
   return (length*width);
   }

int main()
{  double side,radius,length,width;
   printf("Enter side,radius,length,width(one by one): ");
   scanf("%lf%lf%lf%lf", &side,&radius,&length,&width);

   printf("Area of square: %.2lf\n",square_area(side));
   printf("Area of circle: %.2lf\n",circle_area(radius));
   printf("Area of square: %.2lf\n",rectangle_area(length,width));

   return 0;
}
```

Note: (%.2lf) is used to present 2 value after (.)➔24.56
Also can be (%.3lf)➔24.566

❖ **Recursion** : (When a function call itself) ⟶ It is like a loop.

Ex :

```
#include<stdio.h>

void print_name(int count){
    if(count==0)
        return;
    printf("SJTA\n");

    print_name(count-1);

    }

int main()
{   int n;
    printf("Enter the number of times you want to print SJTA:");
    scanf("%d",&n);

    print_name(n);

    return 0;
}
```

Recursive function

Calling itself(will call again and again until count is 0)

- Base case is the condition which stops recursion.
- Iteration(loop) has infinite loop & recursion has stack overflow.(if base condition is absent or not acceptable the program will crash)

▪ **In math**:

f(X)=X^3=X×X×X

(sample of recursion)

let X=2;

□ f(f(f(X)))=f(f(f(2)))=f(f(8))=f(512)

=134,217,728.

➢ **Sum of n natural numbers.** Base value

1 ➔ 1 = sum(1) Base case

2 ➔ 1+2 = sum(1)+2

3 ➔ 1+2+3 = sum(2)+3

4 ➔ 1+2+3+4 = sum(3)+4

(formula)

➔ n ➔ 0+1+2+3+_ _ _ _ _+(n-1)+n = sum(n-1)+n

Code :

```
#include<stdio.h>
int sum(int n){
    if(n==1)
        return 1;

    int sum1= sum(n-1);
    int sum= sum1 + n;
   return sum;
    }
int main(){
    int n;
    printf("Enter the number:");
    scanf("%d",&n);

    printf("Sum is %d\n",sum(n));
    return 0;
}
```

- **Factorial of n natural numbers.**

  ➔5! =1*2*3*4*5

  1! = 1 ➔ factorial(1) Also we know, 0!=1.

  2! = 1*2 ➔ factorial(1)*2

  3! = 1*2*3 ➔ factorial(2)*3

  (n-1)! = (n-1)*(n-2)! ➔ factorial(n-2)*(n-1)

  n! = n*(n-1)! ➔factorial(n-1)*n ➔formula.(when n>1). So, base case➔((n==1)(n==0))➔n!=1

- **Fibonacci sequence** :

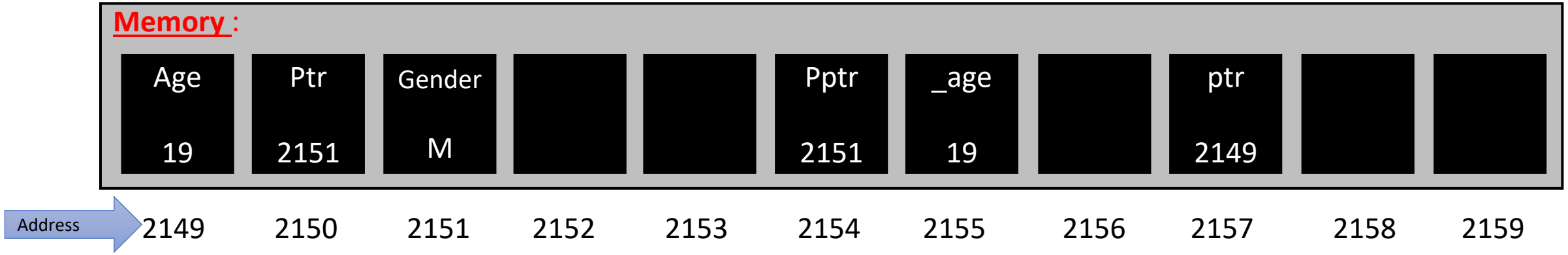
1+1=2

0 , 1 , 1 , 2 , 3 , 5 , 8 , 13 , 21 , 34 , 55 , 89 , 144,……

0+1=1 1+2=3

➔1 to n_{th} Fibonacci sequence = fib(n-1) + fib(n-2) ➔Here, fib(n-1)= fib(n-2) + fib(n-3), means 1 to $(n-1)_{th}$ Fibonacci sequence. And fib(n-2) follows same way. The base case for this program is ➔ if(n==0) return 0; if(n==1) return 1;

❖ **Pointers** : A variable that stores the memory address of another variable.



➔ int Age = 19;
➔ int *ptr = &Age;
➔ char Gender = 'M';
➔ char *Ptr = &Gender;
➔ int _age = *ptr;
➔ int **Pptr = &Ptr;

- (*) ➔ Value of address operator.
- (&) ➔ Address of operator.
- Format specifier: %p ➔ hexadecimal form. %u ➔ unsigned form/normal form.
- Pointer to pointer : A variable that stores the memory address of another pointer.
- ptr, Ptr, Pptr all are variables so any name or word can be use.